Weekly Spotify Wrapped

M122E Final Documentation

1. Project Overview

1.1 Context

Weekly Spotify Wrapped is a Python automation project for module M122E. The program creates a weekly personal music report from Last.fm listening data. It uses Spotify for metadata enrichment, SQLite for local history and Gmail SMTP for email delivery.

1.2 Goal

The goal is to automate a task that would otherwise require several manual steps: checking recent tracks, looking up Spotify information, calculating statistics, creating a report and sending it by email.

1.3 Automated Workflow

- Load configuration from .env.
- Fetch recent tracks from Last.fm.
- Enrich tracks with Spotify metadata where possible.
- Store track plays in SQLite and prevent duplicates.
- Calculate weekly statistics and compare them with the previous week.
- Generate an HTML report.
- Send the report by email if enabled.
- Optionally update a Spotify playlist.

2. How I Worked

I followed the waterfall phases from the assignment. First I clarified the requirements and improved the proposal so that the functional and non-functional requirements were measurable. Then I planned the modules, external systems and database before implementing the script.

During development I tested each part step by step: configuration loading, API clients, database behavior, report generation and email delivery. When Spotify playlist updates returned HTTP 403 because of missing permissions, I kept the playlist feature optional so the main workflow still works reliably.

After the program worked end to end, I improved the email design, added week-over-week comparison and created diagrams and test documentation for the final submission.

3. Project Plan

Phase	Main Work	Result
Requirements Analysis	Define goal, systems and MoSCoW requirements.	Approved and improved project scope.
Design	Plan modules, workflow, APIs and database.	Architecture, UML, flowchart and ER diagrams.
Development	Implement API clients, database, report and email sending.	Working modular Python application.
Testing	Write unit tests and run live workflow checks.	35 pytest tests pass and live email delivery

		was verified.
Documentation	Prepare README, final documentation, proposal and test protocol.	DOCX and PDF documents in docs/.
Deployment	Plan optional weekly execution.	Windows Task Scheduler instructions are included.

4. Requirements

4.1 Functional Requirements

ID	Priority	Requirement	Implementation	Status
FR-01	Must	Load configuration from .env.	config.py	Fulfilled
FR-02	Must	Validate required settings.	config.py, main.py	Fulfilled
FR-03	Must	Fetch Last.fm tracks from the last 7 days.	lastfm_client.py	Fulfilled
FR-04	Must	Store listening data in SQLite.	database.py	Fulfilled
FR-05	Must	Prevent duplicate track plays.	SQLite UNIQUE rule	Fulfilled
FR-06	Must	Calculate statistics.	report_generator.py	Fulfilled
FR-07	Must	Generate an HTML report.	report_generator.py	Fulfilled
FR-08	Must	Send report by Gmail SMTP.	email_sender.py	Fulfilled
FR-09	Should	Enrich tracks with Spotify metadata.	spotify_client.py	Fulfilled
FR-10	Should	Continue if Spotify fails.	main.py	Fulfilled
FR-11	Should	Compare current and previous week.	SQLite history, report_generator.py	Fulfilled
FR-12	Could	Create or update a Spotify playlist.	spotify_setup.py, spotify_client.py	Optional

4.2 Non-Functional Requirements

ID	Category	Requirement	Implementation	Status
NFR-01	Security	No hardcoded secrets.	.env, .env.example, .gitignore	Fulfilled
NFR-02	Reliability	Errors are handled clearly.	custom exceptions and logging	Fulfilled
NFR-03	Maintainability	Code is modular.	separate files per responsibility	Fulfilled
NFR-04	Testability	Core logic is testable.	pytest tests	Fulfilled
NFR-05	Usability	Easy installation and start.	README and Section 7	Fulfilled
NFR-06	Portability	Runs on Windows with Python.	requirements.txt	Fulfilled

5. System Design

5.1 Module Overview

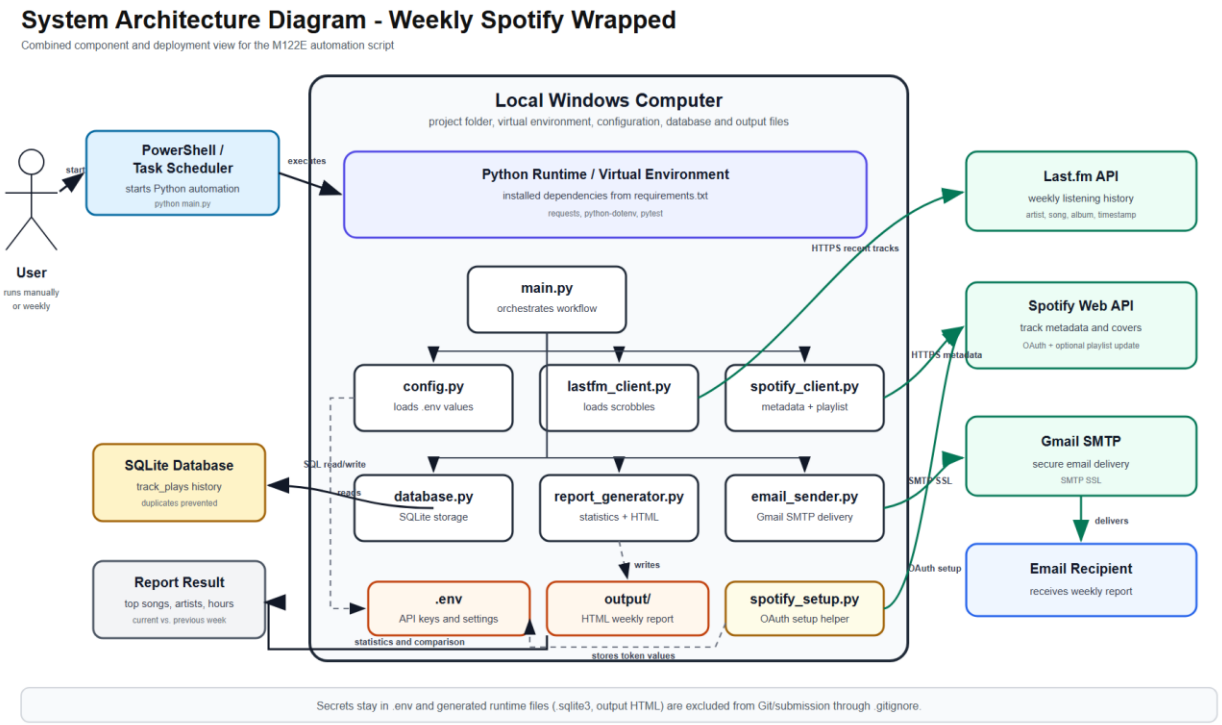
Module	Responsibility
main.py	Starts and coordinates the complete weekly workflow.
config.py	Loads and validates environment variables.
lastfm_client.py	Loads recent track plays from Last.fm.
spotify_client.py	Adds Spotify metadata and optionally updates a playlist.
database.py	Creates and manages the SQLite database.
report_generator.py	Calculates statistics and builds the HTML report.

email_sender.py	Sends the report through Gmail SMTP.
spotify_setup.py	Creates Spotify refresh token and playlist values.

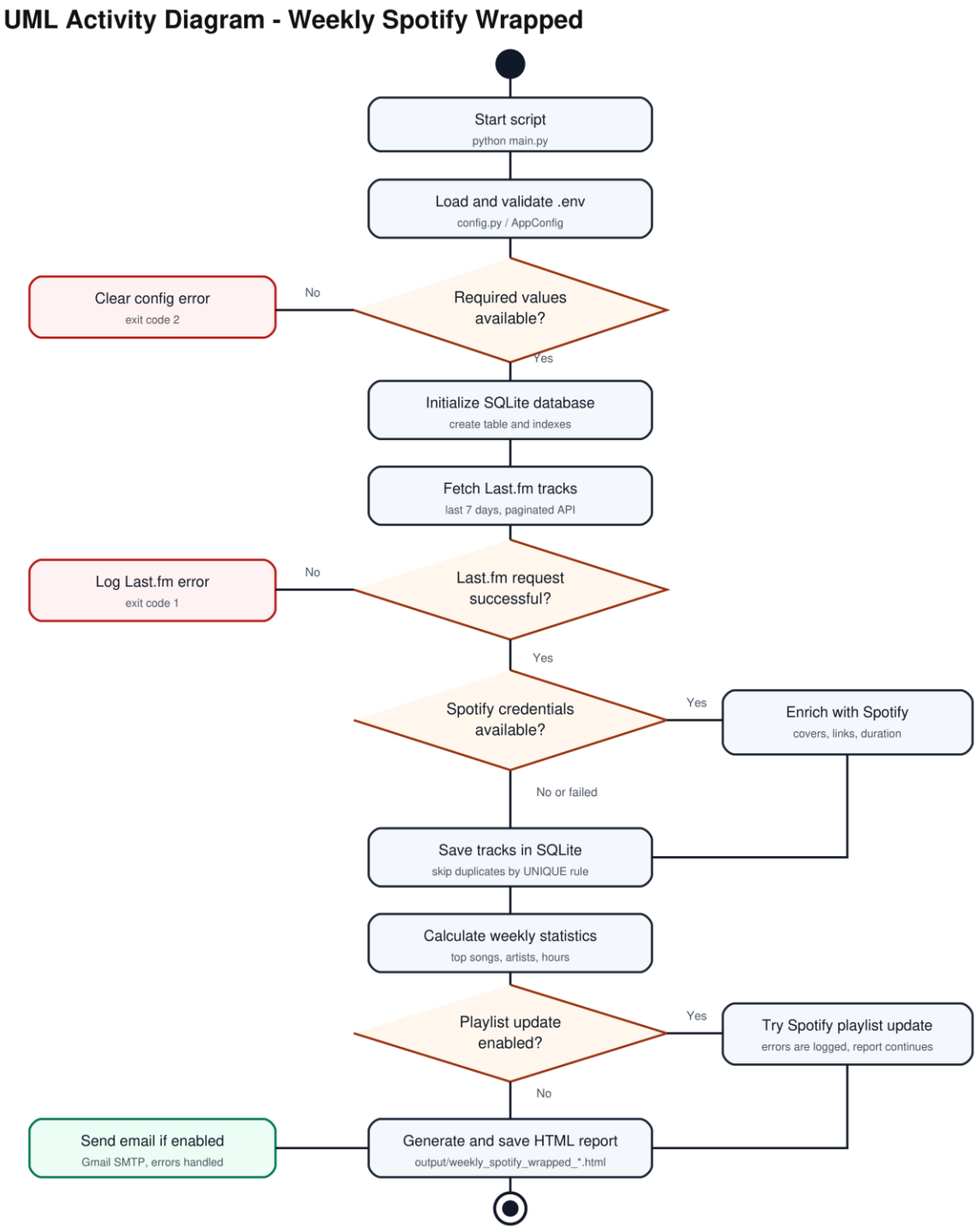
5.2 Main Workflow

1. User or Task Scheduler starts python main.py.
2. Configuration is loaded from .env.
3. Last.fm tracks are fetched.
4. Spotify metadata is added if Spotify is configured.
5. Tracks are saved to SQLite and duplicates are skipped.
6. Current and previous week statistics are calculated.
7. The report is saved as HTML.
8. The report is sent by email if enabled.

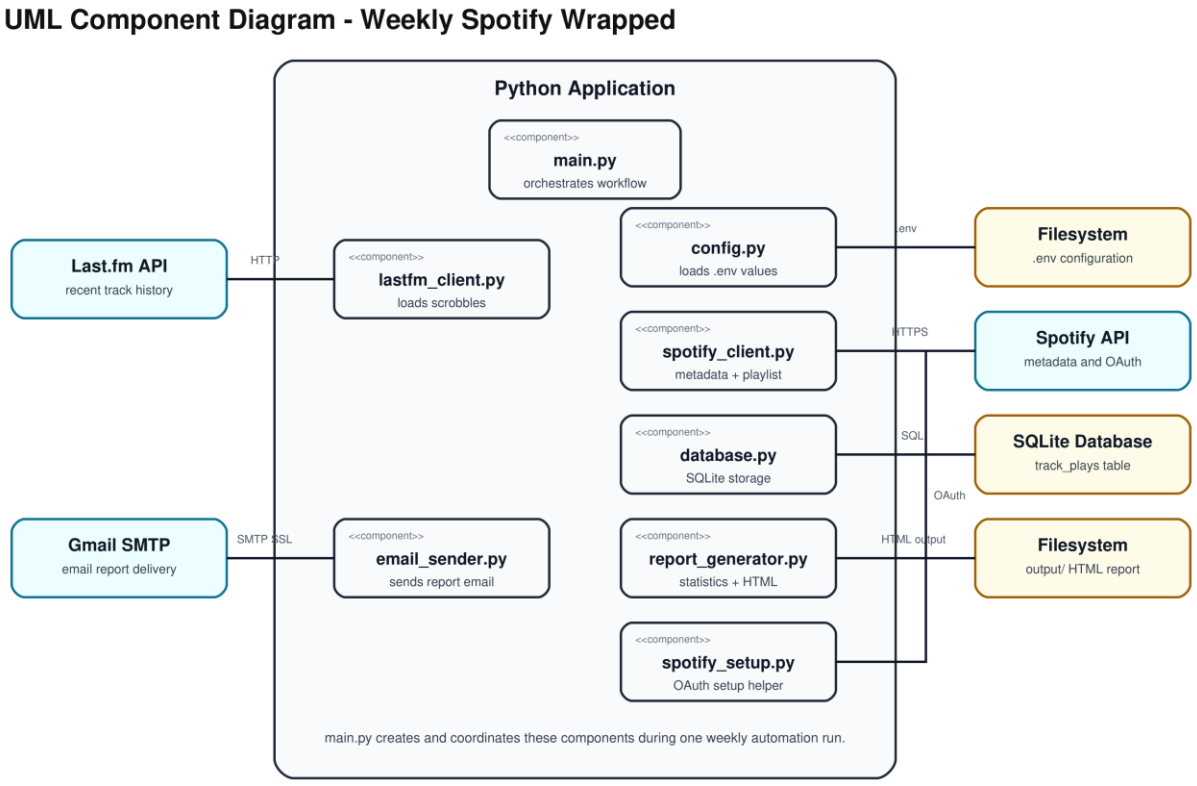
5.3 System Architecture Diagram



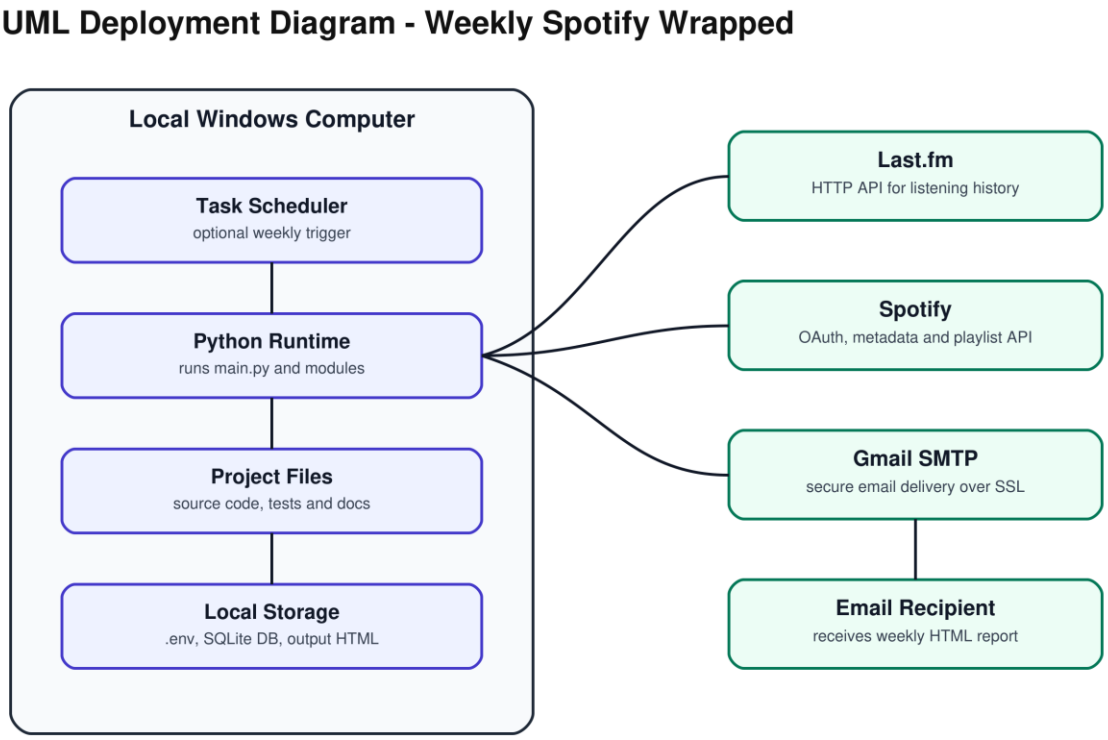
5.4 UML Activity Diagram



5.5 UML Component Diagram

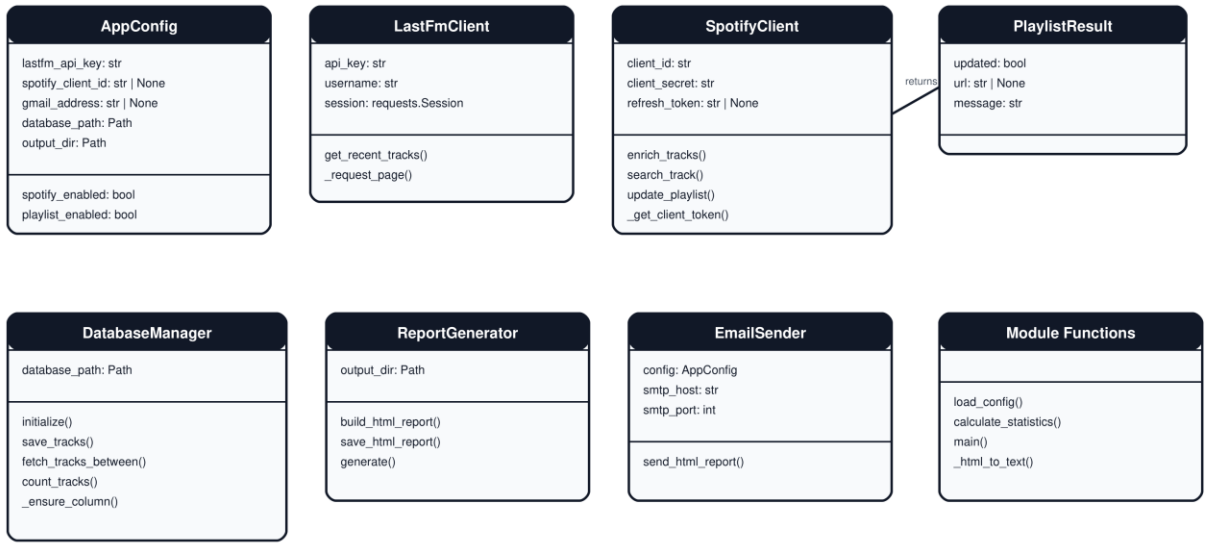


5.6 UML Deployment Diagram



5.7 UML Class Diagram

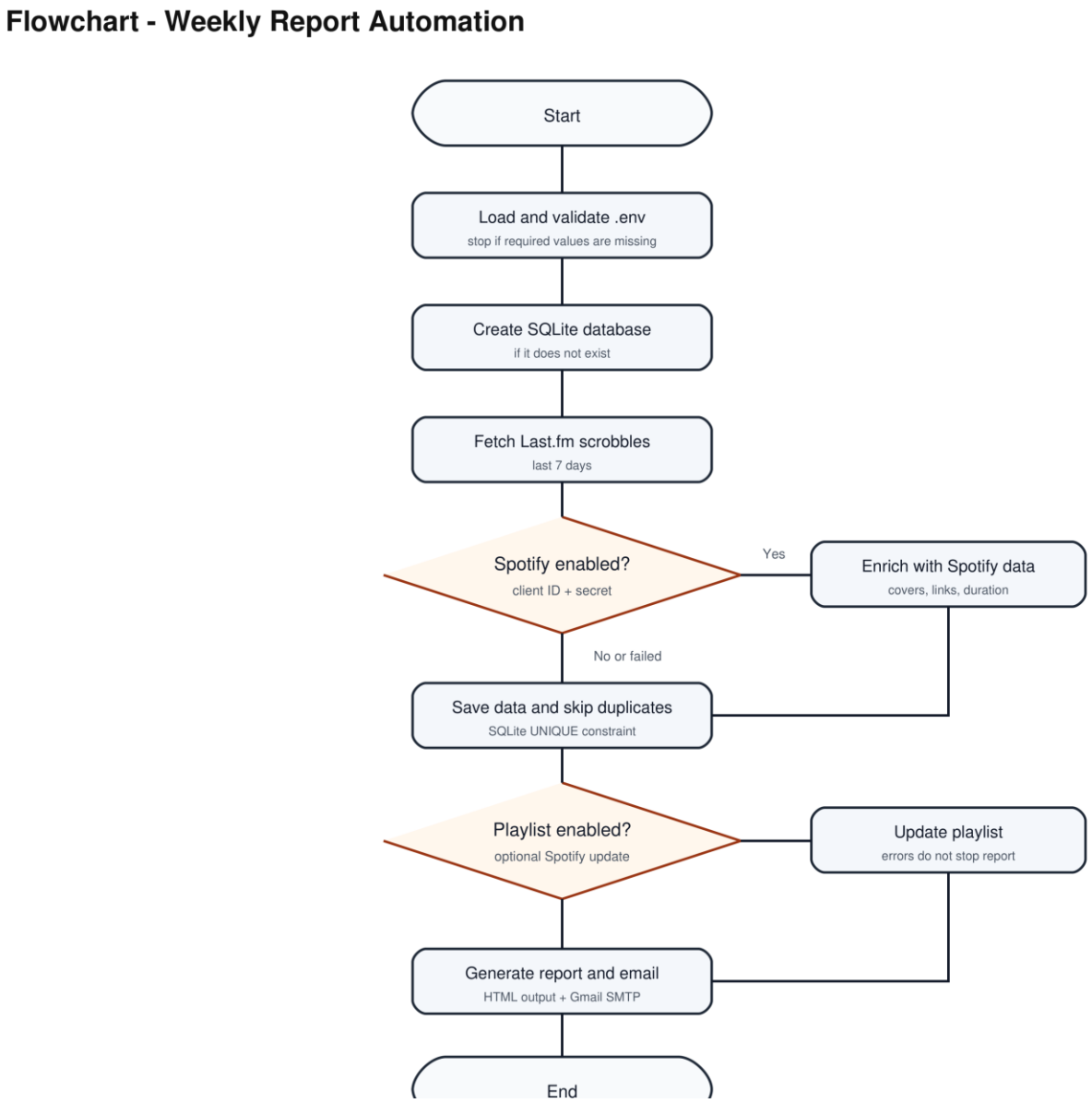
UML Class Diagram - Main Python Structures



main.py creates and coordinates the clients, database manager, report generator and email sender.

5.8 ER Diagram

5.9 Flowchart



6. Database Design

The database uses one main table: track_plays. It stores historical listening data so the program can compare the current week with the previous week.

Column	Purpose
track_name, artist_name, album_name	Music information from Last.fm and Spotify.
played_at	Timestamp of the track play.
lastfm_url	Link to Last.fm track if available.
spotify_track_id, spotify_track_url	Spotify identification and link.
spotify_duration_ms	Track duration used for listening time.
spotify_cover_url, spotify_artist_image_url	Images used in the email report.
created_at	Database insert timestamp.

Duplicate prevention:

```
UNIQUE(track_name, artist_name, played_at)
```

7. Installation And Usage

7.1 Prerequisites

- Windows with PowerShell
- Python installed
- Last.fm account and API key
- Spotify Developer app for optional Spotify features
- Gmail app password if email sending is enabled

7.2 Clone Or Open The Project

```
git clone https://github.com/Svoutat/weekly-spotify-wrapped.git

cd weekly-spotify-wrapped
```

If the project is already stored locally, open PowerShell in the project folder instead.

7.3 Install Dependencies

```
python -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install -r requirements.txt
```

7.4 Configure Environment Values

```
Copy-Item .env.example .env
```

Open .env and fill the required values. The real .env file must never be committed to GitHub.

Value	Purpose
LASTFM_API_KEY	Required Last.fm API key.
LASTFM_USERNAME	Last.fm user whose tracks are loaded.
SPOTIFY_CLIENT_ID / SPOTIFY_CLIENT_SECRET	Spotify metadata enrichment.
SPOTIFY_REFRESH_TOKEN / SPOTIFY_PLAYLIST_ID	Optional playlist update.
GMAIL_ADDRESS / GMAIL_APP_PASSWORD / RECIPIENT_EMAIL	Email sending.
SEND_EMAIL	true sends email, false only creates the HTML report.

7.5 API Setup

Last.fm: create an API application at https://www.last.fm/api/account/create and copy the API key into .env.

Spotify: create an app at https://developer.spotify.com/dashboard, add http://127.0.0.1:8888/callback as redirect URI and run python spotify_setup.py if playlist updates are needed.

Gmail: enable 2-Step Verification, create a Google app password and store it in GMAIL_APP_PASSWORD.

7.6 Start The Program

```
.\.venv\Scripts\Activate.ps1

python main.py
```

The program creates or updates the database, writes the HTML report to output/ and sends the email if enabled.

7.7 Weekly Automation

The script can be scheduled weekly with Windows Task Scheduler.

```
Program:
C:\Users\voutats1\Documents\Gibz\Informatik\M122\M122ESpotifyProject\spotify_weekly_wrapped\.venv\Scripts\python.exe

Arguments: main.py

Start in: C:\Users\voutats1\Documents\Gibz\Informatik\M122\M122ESpotifyProject\spotify_weekly_wrapped
```

8. Testing

Automated tests are executed with pytest.

```
python -m pytest

35 passed
```

The tests cover configuration, database behavior, Last.fm loading, Spotify enrichment, report generation, email sending, Spotify setup and the main workflow. The detailed test protocol is stored as M122E_Test_Protocol.pdf and M122E_Test_Protocol.docx.

9. Assignment Coverage

Assignment Requirement	Evidence In Project
Multiple steps/actions	API loading, enrichment, database, statistics, report, email.
Well-organized scripts	Separate Python modules with clear responsibilities.
Environment variables	.env, .env.example and config.py.
requirements.txt	Included in project root.
Clear start command	python main.py.
Project plan	Excel and PDF project plan in docs/.
Requirements with priorities	Section 4.
Design documents and UML	Section 5 and diagrams folder.
Installation instructions	Section 7 and README.
Executed test protocol	M122E_Test_Protocol.pdf.

10. Conclusion

The project fulfills the M122E assignment because it automates a realistic weekly task with several external systems, stores data locally, uses environment variables, includes tests and contains complete documentation. The code is modular and the diagrams make the workflow and architecture easy to understand.